Lab 6 - TAD: Prática de Pilha

PUC MINAS - CAMPUS CONTAGEM
Disciplina: Algoritmos e Estrutura de Dados
Professor: Fábio Leandro Rodrigues Cordeiro
Alunos: Maria Luiza Mendes Matos e Juan Carlos Barbosa Reis

Questões.

(a) Quantas comparações foram feitas para n = 5? Escreva a expressão geral em função de n e compare com o Bubble Sort clássico sem otimização.

Para n = 5 foram feitas 10 comparações (4 + 3 + 2 + 1 + 0). A expressão geral é:

$C(n) = (n-1) + (n-2) + ... + 1 + 0 = n(n-1)/2$

O Bubble Sort clássico sem otimização também faz $n(n-1)/2$ comparações, então o número é o mesmo e a complexidade continua $O(n^2)$. Isso vale até para um vetor já ordenado, porque o algoritmo não tem parada antecipada (no teste com {1, 2, 3, 4, 5} também deu 10).

(b) Qual é o custo extra de memória em relação ao Bubble Sort com vetor, que ordena no próprio vetor?

O Bubble Sort com vetor ordena no próprio vetor e usa memória extra constante, O(1). A versão com pilhas usa três pilhas (A, B e R), cada uma com capacidade para n elementos, mais a variável carry. O custo extra é O(n), cerca de 3n posições (ou 2n além da cópia dos dados), contra O(1) do vetor.

(c) Que operação do vetor deixamos de ter com a pilha e como o código compensa isso?

A variável carry guarda a “bolha” fora da estrutura, no lugar do elemento que o vetor compararia e trocaria.

Os elementos são desempilhados de A e empilhados em B, na ordem em que a comparação decide.

No fim de cada passada, A e B trocam de papel, e o maior dos restantes vai para R.

(d) Em sua opinião, vale a pena ordenar com pilhas? Em que situação você usaria essa ideia?

Na minha opinião, **não vale a pena** na prática. Gasta mais memória (O(n) extra), exige mais código e não melhora o tempo, que continua O(n²), como no vetor.

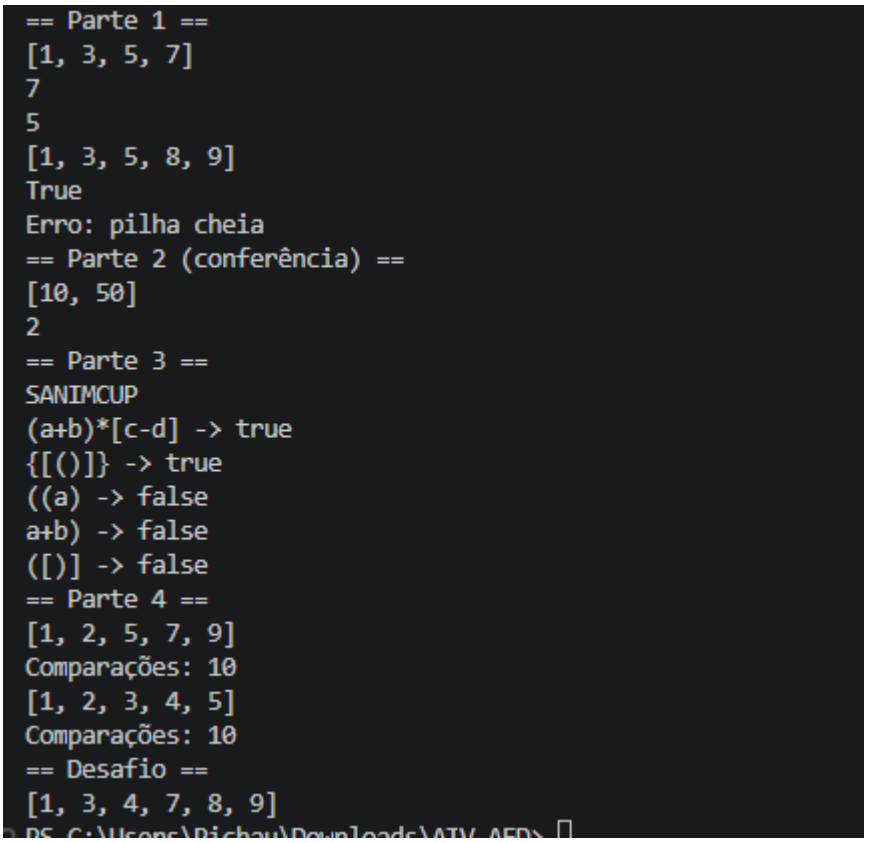
Tabela Parte 2: rastreio

Operação	Retorno	Pilha (base → topo)	topo
empilha(10)	-	[10]	1
empilha(20)	-	[10, 20]	2
desempilha()	20	[10]	1
empilha(30)	-	[10, 30]	2
empilha(40)	-	[10, 30, 40]	3
desempilha()	40	[10, 30]	2
desempilha()	30	[10]	1
empilha(50)	-	[10, 50]	2

Tabela Parte 4.1: rastreio do Bubble Sort com pilhas
Vetor `{5, 2, 9, 1, 7}`

Passada	carry final	B (base → topo)	R (base → topo)
1	9	[1, 7, 2, 5]	[9]
2	7	[2, 5, 1]	[9, 7]
3	5	[1, 2]	[9, 7, 5]
4	2	[1]	[9, 7, 5, 2]
5	1	[]	[9, 7, 5, 2, 1]

Print:


== Parte 1 ==
[1, 3, 5, 7]
7
5
[1, 3, 5, 8, 9]
True
Erro: pilha cheia
== Parte 2 (conferência) ==
[10, 50]
2
== Parte 3 ==
SANIMCUP
(a+b)*[c-d] -> true
{[()]} -> true
((a) -> false
a+b) -> false
([)] -> false
== Parte 4 ==
[1, 2, 5, 7, 9]
Comparações: 10
[1, 2, 3, 4, 5]
Comparações: 10
== Desafio ==
[1, 3, 4, 7, 8, 9]